Codefest 2

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

ECE 410/510

Spring 2026

Overview

This codefest has two sequential challenges. Complete CMAN before opening CLLM. CMAN must be completed without any AI assistance. CLLM permits and often requires AI tooling. Topic: HW/SW co-design and the roofline model. Deliverables are committed to your GitHub repository by the following Sunday at 11:59 pm. Each codefest is graded Satisfactory (S) / Not Yet (NY) as a CMAN+CLLM pair. A late submission counts as Not Yet and consumes one revision token.

CMAN — Roofline construction and kernel classification

Use the following hardware specification: peak compute = 10 TFLOPS (FP32), peak DRAM bandwidth = 320 GB/s, ridge point = 10,000/320 ≈ 31.25 FLOP/byte.

Tasks:

1. On log-log axes (x: FLOP/byte, y: GFLOP/s), draw the roofline: the diagonal bandwidth-limited segment and the flat compute-limited ceiling. Label the ridge point coordinates.
2. Kernel A — Dense GEMM: two FP32 matrices of size 1024×1024 multiplied together. Compute FLOPs ($2\times N^3$ for square matmul), bytes transferred assuming all three matrices (A, B, C) are loaded/stored from DRAM with no cache reuse, arithmetic intensity, and plot the point on your roofline.
3. Kernel B — Vector addition: two FP32 vectors of length 4,194,304 added element-wise. Compute FLOPs, bytes transferred, arithmetic intensity, and plot the point.
4. For each kernel, state: (a) memory-bound or compute-bound on this hardware; (b) attainable performance ceiling in GFLOP/s; (c) what architectural change would most improve performance.

CMAN NO AI	Deliverable: `codefest/cf02/cman_roofline.md` (or .pdf/.png) containing: (a) labeled roofline diagram (hand sketch is fine); (b) GEMM calculation: FLOPs, bytes, AI, attainable GFLOP/s, bound; (c) vector-add calculation same; (d) one-sentence architectural recommendation per kernel with justification.

CLLM — Project algorithm profiling and roofline placement

Profile your chosen project algorithm and place its dominant kernel on a real roofline. Work on your own project algorithm throughout, not a generic benchmark. This session directly produces material for the project milestone M1, which is due Sunday, Apr 12.

Tasks:

5. Profile your project algorithm. Use cProfile or line_profiler to measure execution time per function or loop across at least 10 runs. If your algorithm is a neural network implemented in PyTorch, you may use torch.profiler instead for layer-level timing. Identify the computationally dominant kernel — the one function or loop that accounts for the largest share of total runtime. Save the profiler output as `codefest/cf02/profiling/project_profile.txt` (or .csv).
6. Compute arithmetic intensity for the dominant kernel. Count FLOPs from the arithmetic operations in your kernel (multiplications, additions, etc.). Derive the count analytically from the algorithm, showing the formula and substituted values. If your kernel is a neural network layer, torchinfo or fvcore may assist. Estimate bytes transferred assuming all operands are loaded from DRAM with no reuse. Show the full calculation. Record the result in `codefest/cf02/analysis/ai_calculation.md`.
7. Build a roofline for your target hardware platform (your laptop CPU or GPU). Look up peak FLOP/s and peak memory bandwidth from the spec sheet; compute the ridge point. Plot your dominant kernel on

the roofline. Add a second point representing your hypothetical hardware accelerator design: choose a target compute throughput and on-chip bandwidth and show where it sits. Label all points. Save as `codefest/cf02/profiling/roofline_project.png` (or .pdf).

8. Write a HW/SW partition proposal in `codefest/cf02/analysis/partition_rationale.md` (200–350 words). Address:
    - (a) which kernel(s) you will accelerate in hardware and why the roofline analysis supports that choice;
    - (b) what the software baseline will continue to handle;
    - (c) what interface bandwidth your accelerator would need to avoid becoming interface-bound at the target throughput;
    - (d) whether your kernel is compute-bound or memory-bound on your current hardware, and whether you expect your accelerator design to change that.
9. Update `project/heilmeier.md` with refined answers to all three Heilmeier questions. Your profiling data should now inform question 2 (what are the limits of the current approach?) and question 3 (what is your approach and why is it better?). If the profiling changed your view of the dominant kernel, document that explicitly.

CLLM AI OK	Deliverable: GitHub commit containing `codefest/cf02/profiling/project_profile.txt` (or .csv); `codefest/cf02/analysis/ai_calculation.md` with full FLOPs and bytes calculation shown; `codefest/cf02/profiling/roofline_project.png` (or .pdf) with software kernel and hypothetical HW design both plotted and labeled; `codefest/cf02/analysis/partition_rationale.md` (≥200 words, all four sub-questions addressed); updated `project/heilmeier.md` with all three Heilmeier questions answered. M1 is due Sunday, Apr 12.

S/NY criteria

S/NY criteria	**CMAN — S requires:** roofline diagram present with labeled axes and ridge point; Kernels shown with correct FLOPS. Architectural recommendations consistent with bound classification. **CLLM — S requires:** profiler output present showing per-function or per-layer timing; dominant kernel explicitly identified; arithmetic intensity calculated with FLOPs and bytes shown; roofline present with ridge point labeled and kernel plotted; hypothetical HW design point plotted and labeled on same roofline; `partition_rationale.md` ≥200 words addressing all four sub-questions including bound classification and interface bandwidth estimate; Heilmeier answers updated with data-grounded answers to all three questions. **NY if:** roofline axes not labeled or ridge point missing; arithmetic intensity calculation not shown; no hypothetical HW design point on roofline; partition rationale <100 words; Heilmeier answers not updated.

Appendix: profiling guidance for the three sample algorithms

The three sample algorithms provided with the course are all pure NumPy or plain Python. None uses PyTorch, so torch.profiler does not apply to any of them. Use cProfile for an initial function-level breakdown, then line_profiler on the identified hotspot for line-level timing. The entry points and arithmetic intensity formulas for each algorithm are given below.

CNN with manual backpropagation (cnn_backprop)

The dominant kernel is Conv2D._im2col, which builds the column matrix used by the forward pass convolution. The nested Python loops over output spatial positions make this the clear bottleneck under cProfile.

Profiling entry point:

```
python -m cProfile -s cumtime train.py --data synthetic --epochs 3 | head -30
```

To profile at line level, add @profile to Conv2D._im2col in cnn.py, then run:

```
kernprof -l -v train.py --data synthetic --epochs 1
```

FLOPs for one forward pass through a Conv2D layer (the convolution itself):

FLOPs = $2 \times N \times C_in \times k^2 \times H_out \times W_out \times C_out$

where N is batch size, k is kernel size, and $H_out = H - k + 2p + 1$. The factor of 2 counts one multiply and one accumulate per element. Bytes transferred (no reuse): load the input patch ($N \times C_in \times k^2 \times H_out \times W_out$), the weights ($C_out \times C_in \times k^2$), and store the output ($N \times C_out \times H_out \times W_out$), each in FP64 (8 bytes).

Transformer language model (transformer_lm)

The dominant kernel is the multi-head self-attention forward pass, specifically the Q, K, V projection matrix multiplications and the scaled dot-product attention score computation. These are the heaviest operations per token and scale quadratically with sequence length.

Profiling entry point:

```
python -m cProfile -s cumtime train.py --steps 100 | head -30
```

To zoom in on the attention forward function, add @profile to the attention_forward function in transformer.py and run with kernprof.

FLOPs for one transformer layer, one forward pass (batch size B, sequence length T, model dimension d, feed-forward dimension d_ff, number of heads h):

Attention projections (Q, K, V, O): $4 \times 2 \times B \times T \times d^2$

Attention scores (QK^T): $2 \times B \times h \times T^2 \times (d/h)$

Attention weighted sum (AV): $2 \times B \times h \times T^2 \times (d/h)$

Feed-forward (two matmuls): $2 \times 2 \times B \times T \times d \times d_ff$

Bytes transferred: load all weight matrices ($4d^2 + 2 \times d \times d_ff$ parameters, each 8 bytes for FP64), load input activations ($B \times T \times d \times 8$ bytes), store outputs.

Tabular Q-learning (qlearning_gridworld)

Q-learning is not a neural network. Its compute structure is fundamentally different: there are no matrix multiplications. The dominant cost is the training loop itself — specifically QLearningAgent.update (and _update_standard), called once per environment step across millions of episodes. Each update is a handful of scalar operations on a Q-table indexed by state and action.

Profiling entry point:

```
python -m cProfile -s cumtime train.py --episodes 20000 | head -30
```

FLOPs per Q-table update (standard Q-learning):

The update rule is: $Q[s,a] \leftarrow Q[s,a] + \alpha \times (r + \gamma \times \max Q[s'] - Q[s,a])$. This requires one argmax over n_actions values, two subtractions, two multiplications, and one addition: approximately $2 \times n_actions + 5$ FLOPs per step. For a 4-action gridworld that is roughly 13 FLOPs per step.

Bytes transferred per update: read Q[s, :] (n_actions × 8 bytes for FP64) and Q[s′, :] (same); write one element back. For n_actions = 4 that is 72 bytes per update, giving an arithmetic intensity of roughly 13/72 ≈ 0.18 FLOP/byte — deeply memory-bound even by CPU standards. This is the key insight for the roofline: hardware acceleration of tabular Q-learning is limited by the random Q-table lookups, not by arithmetic.